# MLVU information sheet

*Please include this page in your report either at the start or at the end, before the appendix. Do not change the formatting.*

**Group number**

156

**Authors**

| name | student number |
|---:|---:|
| Conrad Clough | 2699315 |
| Samuel Power | 2800107 |
| ANDI FILL THIS OUT :) | THIS TOOOOO |

**Software used** *Describe briefly which software and libraries you relied on. If you built a particular algorithm from scratch, make sure to state it explicitly here.*

We used Jupyter, numpy, pandas, matplotlib, and sklearn. Jupyter was used for ease of modifying our program without needing to redo data processing; numpy, pandas, and matplotlib were used for interacting with data and displaying results; sklearn was used for their model implementations.

**Use of AI tools** *If you used tools like ChatGPT or Github Copilot in any phase of the report, please detail here which tools you used, what parts of the work you used them for and how. Note that if you use an AI tool and don't report it, it's considered fraud, so be detailed. Note also that having an AI tool write for you is not allowed. If no AI tools were used, please state that explicitly.*

- Conrad: ChatGPT was used to generate the code that created our graphs, specifically the styling.

**Link to code (optional)** https://github.com/Flootersmm/ml_project

**Group disagreements (optional)** *If there are any disagreements in the group, you may **not** remove a student from the author list without their consent. You should bring disagreements to our attention early. As a last resort, you can describe any grievances here.*

# How can machine learning-based DDoS detection models be designed to minimise false positive rates while maintaining high detection accuracy on both low- and high-powered hardware?


Conrad Clough
*VUNetId: cch250*
*Student Nr: 2699315*
c.j.clough@student.vu.nl

Samuel Power
*VUNetId: spo237*
*Student Nr: 2800107*
s.j.power@student.vu.nl

mr. andy
*VUNetId: x*
*Student Nr: x*
lovelyman@student.vu.nl

# I. Abstract

Distributed Denial of Service (DDoS) attacks continue to disrupt network services, causing downtime and economic damage. Existing techniques for mitigating this often struggle to whatever they struggle with, find a paper and cite it. This paper investigates the design of DDoS detection models using machine learning. We evaluate three classifiers, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forests, by analysing their accuracy, false positive rates, and resource usage under controlled conditions. Our results indicate that whatever we find. One sentence about each, what they do well and what they do bad. These findings suggest that model selection is dependent on hardware specifications and the relative importance of minimising false positives, providing a practical guide for using machine learning-based DDoS detection in diverse contexts.

**Keywords**: DDoS detection, machine learning, false positive rate, k-Nearest Neighbours, Random Forest, Logistic Regression, computational efficiency

# II. Introduction

The proliferation of online services has made networks increasingly vulnerable to Distributed Denial of Service (DDoS) attacks, which can overwhelm servers and disrupt them, causing significant operational downtime and economic damage. Detecting these attacks efficiently is challenging because high accuracy must be balanced with low false positive rates; denying service to legitimate users is highly undesirable, as is allowing too much malicious traffic through. Machine learning offers a promising alternative to traditional, rule-based approaches; patterns in network traffic can be used to identify evolving malicious activity in real time. This document investigates the design of machine learning-based DDoS detection models with an emphasis on maintaining high detection accuracy while reducing false positives, and evaluates their resource usage across a range of hardware to ensure usability for a variety of situations. Specifically, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forest classifiers are compared in terms of both accuracy, false positive rate, and performance.

Blah blah blah. We can quote code `like this`. We can also reference sections like this Section II or even define labels on things like figures and images and reference them later.

Wanna reference something in the bibliography? Just do it with another label [1]

The report is structured as follows:

- In Section III we consider existing methods of DDoS detection.
- In Section VIII we discuss the dataset we used and how it was prepared for use in training.
- In Section IX we explain our classifiers and how we ran our experiments.
- In Section IV we show our results.
- In Section V we interpret our results and address what they mean.
- In Section VI we summarise our findings.

TABLE I
Resource usage for model training on desktop, with 200 000 samples.

| Model | Hardware | RAM Δ (MB) | CPU Time (s) | Wall Time (s) |
|---|---|---|---|---|
| kNN | Desktop | 2034.49 | 10121.29 | 1163.05 |
| RF | Desktop | 2358.29 | 1619.48 | 116.39 |
| LR | Desktop | 233.83 | 69.41 | 42.31 |

# III. Related Work

# IV. Results

# V. Discussion

# VI. Conclusion

knn does worse on imbalanced because the larger group sizes 'pull' knn into them

find a paper about this

discuss how you can change this, with k tuning or with weights on the classes that represent the imbalance

# VII. Related Work stuff

TO MERGE TOGETHER LATER

Previously, rule-based classifiers were used to detect DDoS attacks [2]. This involves the categorisation data with a series of "if … then …" rules, generating descriptive models that are easy to interpret. This is an earlier machine learning method than the classifiers explored in this report, and therefore less expressive. Requiring distinct rules that must be carefully considered to provide good coverage and classification makes this kind of model perform poorly in exceptional scenarios. Real-world data is often fuzzy, and preventing DDoS attacks is an arms race - the complexity and upkeep of a rule-based classifier fighting new attacks with evolving signatuers makes them unwieldy.

There are many ways to use kNN, and one notable study reduces the feature space drastically to only four features [3]. In constrast, this report applies kNn to all numeric features available in the dataset, a total of 79, to evaluate whether broader feature coverage improves performance or not.

Other papers have investigated the usage of a kNN model to tackle DDoS attacks in real-time. It's correctly pointed out in [4] that machine learning is a rapidly developing field, and therefore repeated investigation into model performance is important and relevant to consider. The author expressed interest in investigating the computational resource requirements for this real-time protection; resource-constrained IoT environments were less performant than desired and required further optimisation to reach a high level of effectiveness.

# VIII. Dataset and Preprocessing

Three models were trained on a dataset consisting of network traffic, balanced between DDoS traffic and benign traffic. These models were kNN, logistic regression, and random forest. The data was extracted from public IDS datasets, produced in different years and with different techniques of

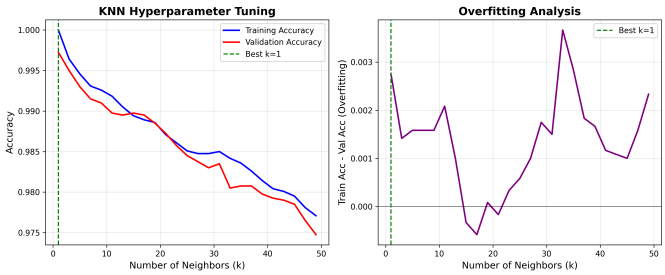


Fig. 1. Tuning hyperparameters for kNN for accuracy.

generating traffic. There are 12 794 627 datapoints and 84 features. The models' inference abilities were then tested on an identically formed dataset that was unbalanced, with 20% DDoS traffic and 80% benign. Implementation of each model was provided by the `sklearn` library for Python.

In order to manage resource and time constraints, training and inference were limited to 200 000 labels. This data was then put into a `pandas` dataframe for efficiency and ease of management. Particular features that were either irrelevant or non-numeric were also dropped - both kNN and logistic regression require numeric features. An alternative way of tackling non-numeric features is to one-hot encode them, where categoric features are given an integer identifier. This massively increases the memory requirements of the model and was therefore decided against, with the exception of encoding the `Protocol` feature which was the only relevant categorical feature present. This resulted in 79 features.

The next step was cleaning; all features were represented with a 64-bit float and invalid numbers were cast to NaN for predictable management.

Next, the dataset was split into 60% training, 20% validation, and 20% test data. The training set was used for fit the model parameters, and was the largest to allow the models to effectively capture patterns in the data. The validation set was used for hyperparameter tuning; as the models haven't encountered the data before this is a suitable playground for adjustment. The final test set allows finalisation of ideal hyperparameters, and an additional verification that they are indeed ideal without any bias.

Finally, features were then scaled with a mean of 0 and a standard deviation of 1, which is z-score normalisation. This is because logistic regression is sensitive to feature magnitude as it uses gradient descent which can struggle when, for example, one feature takes values $[0, 1]$ and another takes $[2^{32}, 2^{64}]$. This prevents particular features completely dominating calculations and having inflated importance. It also allows faster convergence to the gradient's local minima. This scaler was applied to the training data, and then used to transform all data to make it homogeneous for the models. This is the last step to prepare the data.

Exact implementation details can be viewed at the codebase's git repository [1].

## IX. Methodology

### A. kNN

The first model evaluated was kNN. This works by finding the $k$-nearest points in the training dataset and uses their class to predict the class of nearby datapoints. For 79 features, the feature space has a dimensionality of $\mathbb{R}^{79}$ which makes visualisation challenging, but the idea of 'nearby datapoints' generlises to high-dimension Euclidean spaces and therefore kNN is perfectly usable, though potentially slow REPHRASE, CITE, OR REFERENCE HYPOTHESIS. Varying hyperparameters $k$ were tested to maximise accuracy and minimise overfitting, see Fig. 1. Larger $k$ can lead to overfitting and smaller $k$ can lead to underfitting, so finding the balance was important.

After the optimal hyperparemter $k$ was chosen, 5-fold cross-validation was used, wherein the validation and training sets were combined and split into 5 sections. This allows for addressing the tradeoff between bias and variance, finding a model that maximises the benefit gained for the best inference results. Finally, the model was trained and then evaluated a single time on the test data. Using the test data a single time to evaluate final performance is extremely important to prevent provide an independent analysis of the model's performance - repeated use of the test data means this is no longer an evaluation on unseen data and therefore valueless.

### B. Logistic Regression

### C. Random Forest

Once the models were evaluated, we created confusion matrices, ROC curves, and computed AUC scores for each of our models. The models themselves were also saved to later be used for inference. This inference was performed with identical dataset preprocessing on the unbalanced dataset. Confusion matrices, ROC curves, and AUC scores were also computed for the inference run, and only one run was performed per model as the results are deterministic.

Throughout both training and analysis, performance statistics were collected. These were RAM usage, CPU time, and wall clock time. Training was done on a desktop computer equipped with a 12th Gen Intel(R) Core(TM) i7-12700K and 32GiB of DDR5 4800MHz RAM, and inference was ran on both the same desktop computer and a Raspberry Pi 4 with a Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz and 4GiB of DDR4 3200MHz RAM. This was done to measure performance under limited hardware.

## X. Results (kNN, to merge)

### A. Training Overhead

Reference table 1

Training costs are a one-time expense, but they're relevant when considering that models need to be occasionally retrained as attacks evolve and signatures change. Briefly compare results, it's okay to essentially just rephrase what the table says. A significant portion of the training time for kNN comes from the 5-fold cross-validation: having to compute

TABLE II
RESOURCE USAGE FOR MODEL INFERENCE ACROSS HARDWARE, EVALUATED ON 200 000 SAMPLES.

| Model | Hardware | RAM Δ (MB) | Wall Time (s) | Through-put (flows/s) | Latency (ms/flow) |
|---|---|---|---|---|---|
| kNN | Desktop | 60.95 | 141.90 | 1409 | 0.710 |
| kNN | Raspberry Pi 4 | 290.61 | 1408.23 | 142 | 7.041 |
| RF | Desktop | 6.48 | 7.95 | 25145 | 0.040 |
| RF | Raspberry Pi 4 | 64.94 | 65.15 | 3070 | 0.326 |
| LR | Desktop | 36.73 | 7.42 | 26943 | 0.037 |
| LR | Raspberry Pi 4 | 68.63 | 30.79 | 6497 | 0.154 |

the distances between 200 000 samples is incredibly slow. For larger datasets and more thorough validation, kNN may be completely unrealistic as an option.

### B. Inference Throughput

These figures

## References


[1] A. S. C. Clough S. Power, "Project Codebase on Github." [Online]. Available: https://github.com/Flootersmm/ml_project

[2] S. K. S. P. Y. e. a. Rajendran R., "Detection of DoS attacks in cloud networks using intelligent rule based classification system.," *Cluster Computing*, vol. 22, no. ?_number, pp. 423–434, Feb. 2019, doi: 10.1007/s10586-018-2181-4.

[3] S. Dong and M. Sarem, "DDoS Attack Detection Method Based on Improved KNN With the Degree of DDoS Attack in Software-Defined Networks," *IEEE Access*, vol. 8, no. , pp. 5039–5048, 2020, doi: 10.1109/ACCESS.2019.2963077.

[4] A. I. Gide and A. A. M. zu, "A Real-Time Instrusion Detection System for DoS/DDoS Attack Classification in IoT Networks Using KNN-Neural Network Hybrid Technique," *Babylonian Journal of Internet of Things*, vol. 2024, no. , pp. 60–69, 2024, doi: 10.58496/BJIoT/2024/008.